POLS 602 — Homework 1: When Do Estimates Miss?

Aryan Jignesh Anand

Problem 1: Why divide by n − 1?

Given: The population has mean 0, standard deviation 2, and variance 4.

(a)

```
estimates = function(x) {
  n = length(x)                          # Number of observations
  squared_deviations = (x - mean(x))^2   # Squared distances from sample mean
  variance_n = sum(squared_deviations) / n
  variance_n1 = var(x)                   # R divides by n - 1
  sample_sd = sd(x)                      # Square root of var(x)
  return(c(variance_n = variance_n,
           variance_n1 = variance_n1, sample_sd = sample_sd))
}

set.seed(60201)  # Fixing the random numbers so results can be reproduced.
x10 = rnorm(10, mean = 0, sd = 2)
cat("\nPROBLEM 1(a): sample of 10 and its three estimates\n")


PROBLEM 1(a): sample of 10 and its three estimates

print(x10)

 [1] -3.7330521  1.3626286  2.3321504 -1.9003909  0.6015415  2.4851096
 [7]  0.6475149  2.4809134 -1.5766448 -2.6824261

print(estimates(x10))

 variance_n variance_n1   sample_sd
   4.763586    5.292874    2.300625

print(all.equal(unname(estimates(x10)[2]),
                sum((x10 - mean(x10))^2) / (length(x10) - 1)))

[1] TRUE

print(all.equal(unname(estimates(x10)[3]), sqrt(var(x10))))

[1] TRUE
```

Interpreting the function:

The function `estimates(x)` returns three numbers:

- `variance_n`: the sum of squared deviations from the sample mean, divided by n.
- `variance_n1`: `var(x)`, which divides that sum by $n - 1$.
- `sample_sd`: `sd(x)`, which is the square root of `var(x)`.

For the simulated sample of 10, the estimates are:

Divide-by-n variance	Divide-by-(n−1) variance	Sample SD
4.7636	5.2929	2.3006

(b)

```
reps = 1000    # Starting with 1,000 empty rows. Each row will hold one sample's estimates.
set.seed(60202)

results5 = data.frame(variance_n = rep(NA, reps),
                      variance_n1 = rep(NA, reps),
                      sample_sd = rep(NA, reps))
for (i in 1:reps) {
  x = rnorm(5, mean = 0, sd = 2)
  results5[i, ] = estimates(x)
}
cat("\nPROBLEM 1(b): first six rows (one row per sample)\n")


PROBLEM 1(b): first six rows (one row per sample)

print(head(results5))

  variance_n variance_n1 sample_sd
1   3.396779    4.245974  2.060576
2   4.087573    5.109466  2.260413
3   5.234832    6.543540  2.558034
4   1.720155    2.150193  1.466354
5   2.665125    3.331406  1.825214
6   1.591406    1.989257  1.410410

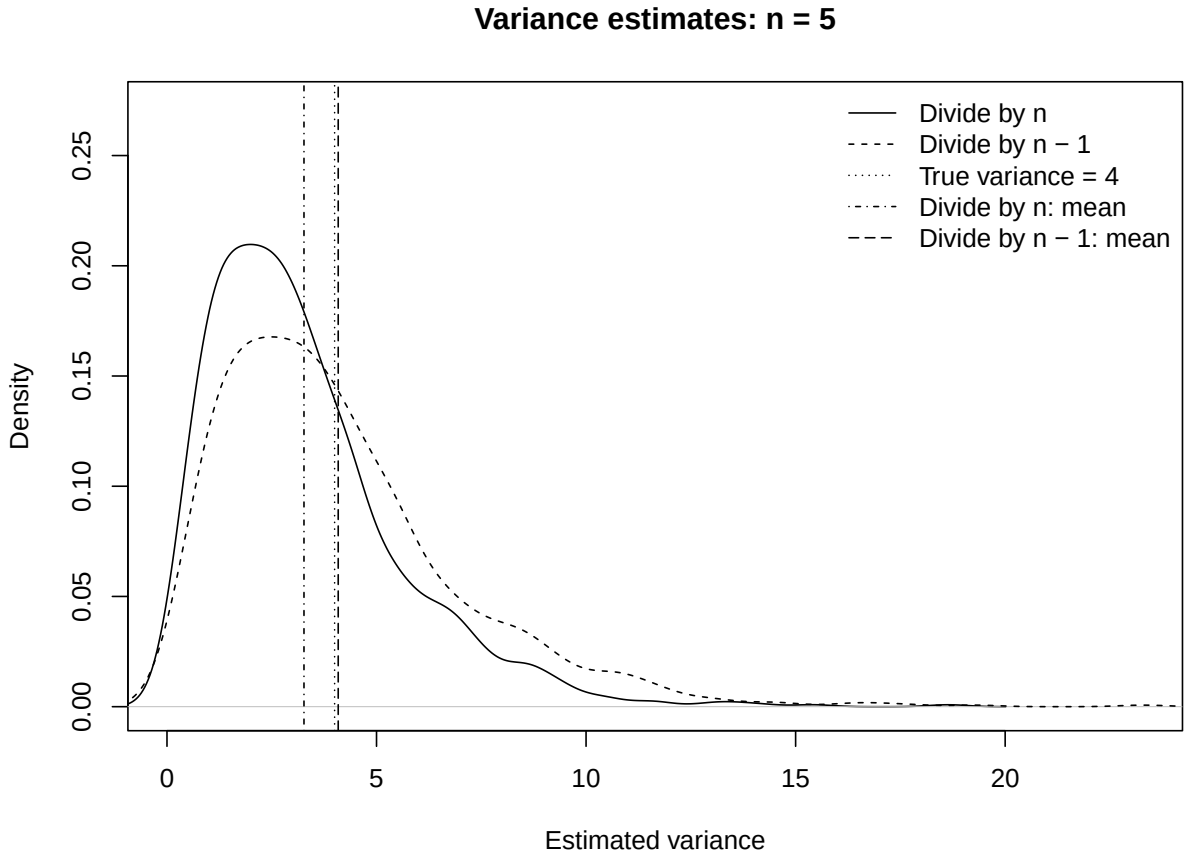
cat("Means of the two variance estimators:\n")

Means of the two variance estimators:

print(c(divide_by_n = mean(results5$variance_n),
        divide_by_n1 = mean(results5$variance_n1)))

 divide_by_n divide_by_n1
    3.268854     4.086068
```

```
plot_variances = function(results, n) {
  d_n = density(results$variance_n)
  d_n1 = density(results$variance_n1)
  plot(d_n, xlim = c(0, max(results$variance_n1)),
       ylim = c(0, max(d_n$y, d_n1$y) * 1.3),
       main = paste("Variance estimates: n =", n),
       xlab = "Estimated variance", ylab = "Density")
  lines(d_n1, lty = 2)
  abline(v = 4, lty = 3)
  abline(v = mean(results$variance_n), lty = 4)
  abline(v = mean(results$variance_n1), lty = 5)
  legend("topright", legend = c("Divide by n", "Divide by n - 1",
         "True variance = 4", "Divide by n: mean",
         "Divide by n - 1: mean"), lty = 1:5, bty = "n")
}
plot_variances(results5, 5)
```



Sampling distributions with n = 5

The data frame `results5` contains 1,000 rows, one for each independent sample, and three columns containing the estimates.

Estimator	Mean of 1,000 estimates	Estimated bias: mean minus 4
Divide by n	3.2689	−0.7311
Divide by n − 1	4.0861	+0.0861

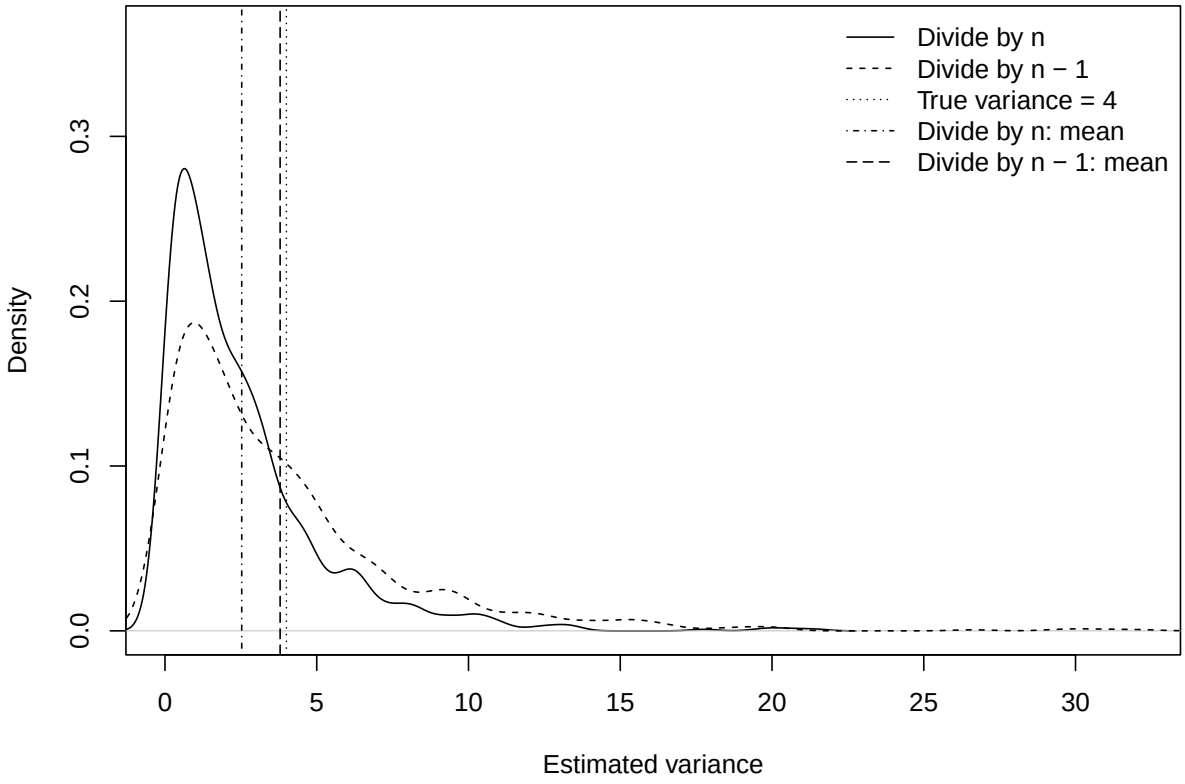
The divide-by-n estimator is **downward biased**: it systematically underestimates the population variance. The divide-by-(n−1) estimator is unbiased in theory. Its simulated mean is not exactly 4 because 1,000 repetitions still contain Monte Carlo randomness.

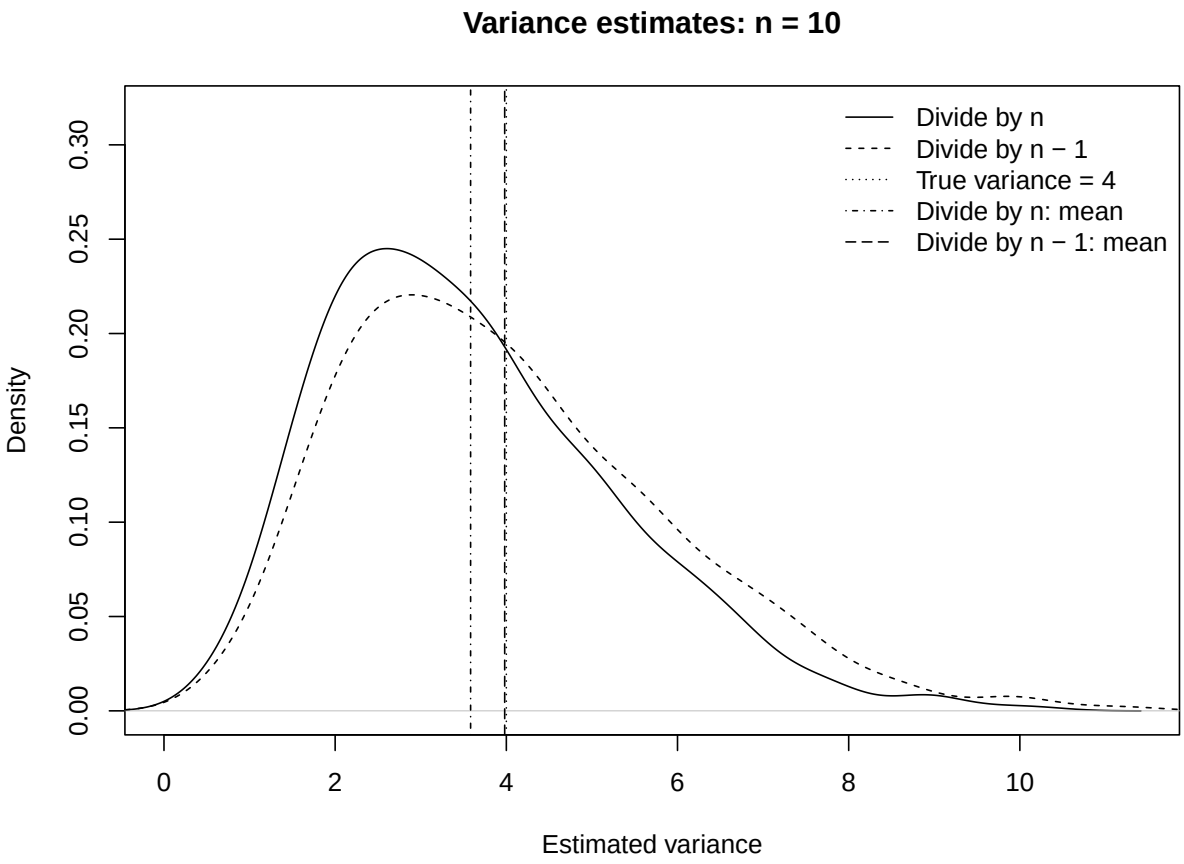
The distribution plot uses a solid line for divide-by-n estimates and a dashed line for divide-by-(n−1) estimates. Other line styles mark the true variance and each estimator's simulated mean, as shown in the legend. At n = 5 the divide-by-n distribution is shifted toward smaller values and is less spread out.

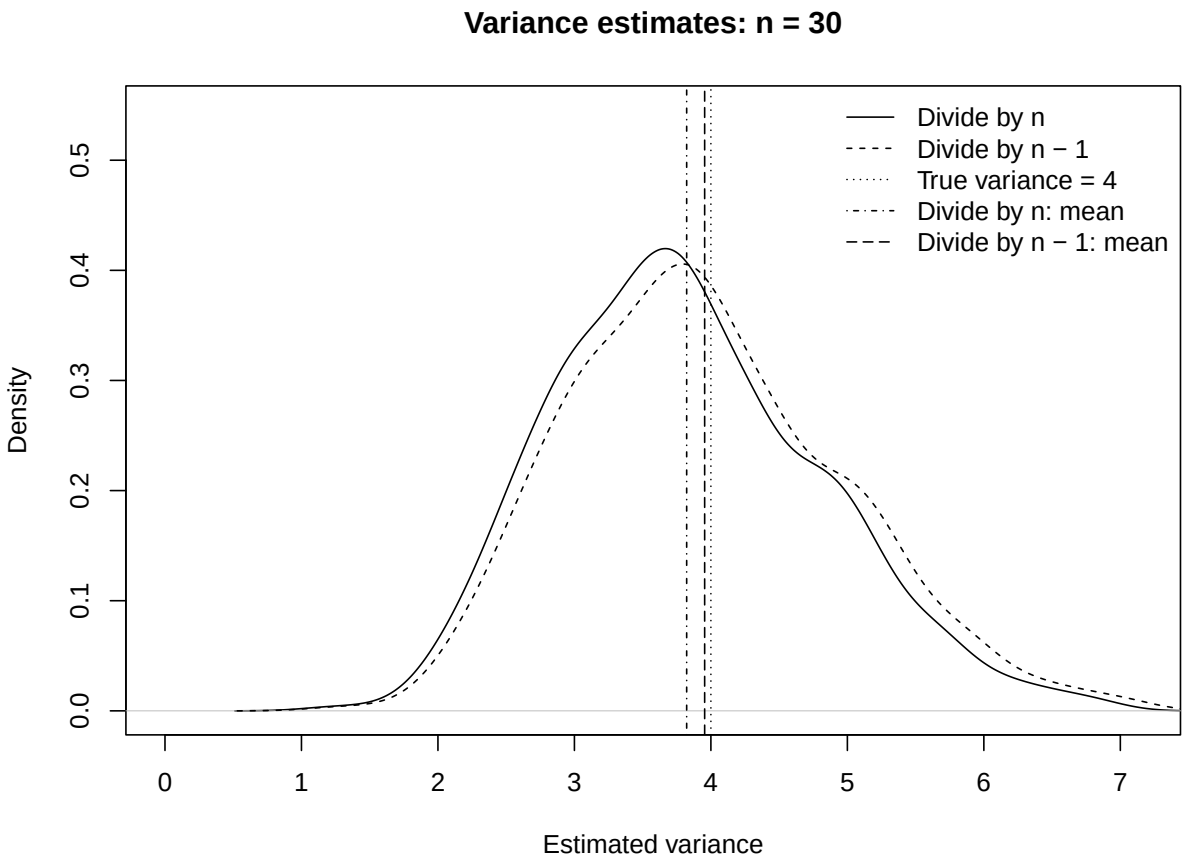
(c)

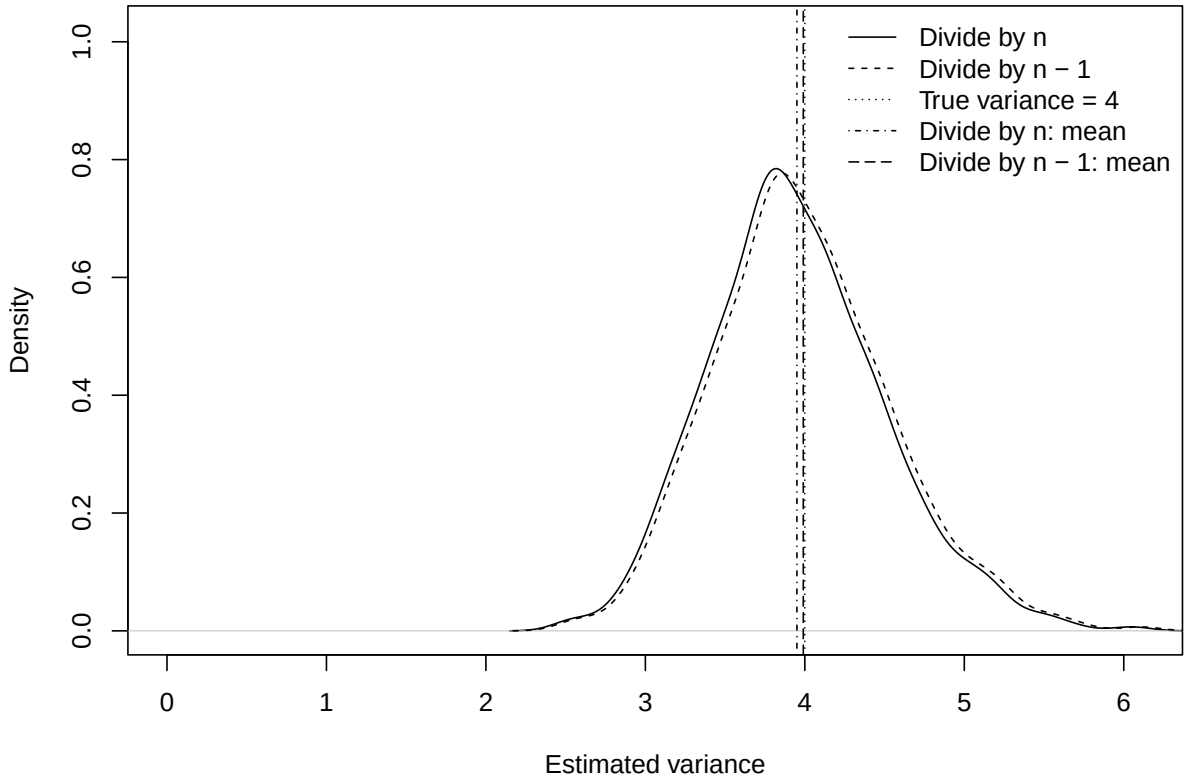
```
sample_sizes = c(3, 5, 10, 30, 100)

bias_table = data.frame(n = sample_sizes, mean_var_n = NA,
                        mean_var_n1 = NA, bias_var_n = NA,
                        bias_var_n1 = NA, mean_sd = NA, bias_sd = NA)
set.seed(60203)
for (j in 1:length(sample_sizes)) {
  n = sample_sizes[j]
  if (n == 5) {
    # Reusing part (b), so all answers referring to n = 5 are consistent.
    results = results5
  } else {
    results = data.frame(variance_n = rep(NA, reps),
                         variance_n1 = rep(NA, reps),
                         sample_sd = rep(NA, reps))
    for (i in 1:reps) {
      x = rnorm(n, mean = 0, sd = 2)
      results[i, ] = estimates(x)
    }
    plot_variances(results, n)
  }
  bias_table$mean_var_n[j] = mean(results$variance_n)
  bias_table$mean_var_n1[j] = mean(results$variance_n1)
  bias_table$bias_var_n[j] = mean(results$variance_n) - 4
  bias_table$bias_var_n1[j] = mean(results$variance_n1) - 4
  bias_table$mean_sd[j] = mean(results$sample_sd)
  bias_table$bias_sd[j] = mean(results$sample_sd) - 2
}
```


Variance estimates: n = 3
Divide by n
Divide by n − 1
True variance = 4
Divide by n: mean
Divide by n − 1: mean
Density
0.0
0.1
0.2
0.3
0
5
10
15
20
25
30
Estimated variance


Variance estimates: n = 10
Divide by n
Divide by n − 1
True variance = 4
Divide by n: mean
Divide by n − 1: mean
Density
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0
2
4
6
8
10
Estimated variance


Variance estimates: n = 30
Divide by n
Divide by n − 1
True variance = 4
Divide by n: mean
Divide by n − 1: mean
Density
0.0
0.1
0.2
0.3
0.4
0.5
0
1
2
3
4
5
6
7
Estimated variance



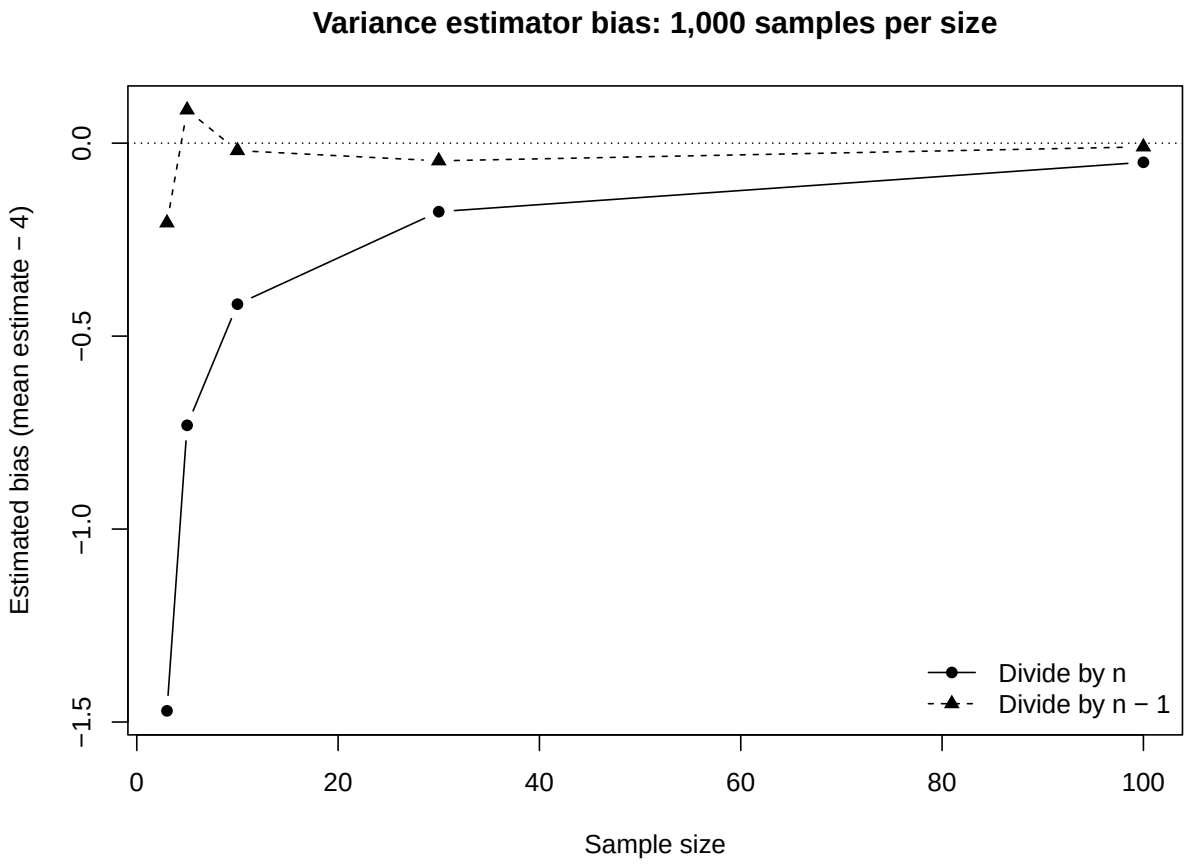

```
cat("\nPROBLEM 1(c) and 1(d): simulation means and estimated biases\n")
```

```
PROBLEM 1(c) and 1(d): simulation means and estimated biases
```

```
print(round(bias_table, 4), row.names = FALSE)
```

```
   n mean_var_n mean_var_n1 bias_var_n bias_var_n1 mean_sd bias_sd
   3     2.5290      3.7935    -1.4710     -0.2065  1.7131 -0.2869
   5     3.2689      4.0861    -0.7311      0.0861  1.9018 -0.0982
  10     3.5827      3.9807    -0.4173     -0.0193  1.9392 -0.0608
  30     3.8224      3.9543    -0.1776     -0.0457  1.9723 -0.0277
 100     3.9504      3.9903    -0.0496     -0.0097  1.9928 -0.0072
```

```
plot(bias_table$n, bias_table$bias_var_n, type = "b", pch = 16,
     ylim = range(c(bias_table$bias_var_n,
                                bias_table$bias_var_n1, 0)),
     xlab = "Sample size", ylab = "Estimated bias (mean estimate - 4)",
     main = "Variance estimator bias: 1,000 samples per size")
lines(bias_table$n, bias_table$bias_var_n1, type = "b", pch = 17,
      lty = 2)
abline(h = 0, lty = 3)
legend("bottomright", legend = c("Divide by n", "Divide by n - 1"),
       pch = c(16, 17), lty = c(1, 2), bty = "n")
```



Changing the sample size

Each size uses 1,000 samples. The n = 5 results from part (b) are reused.

n	Estimated bias, divide by n	Estimated bias, divide by n − 1	Theoretical bias, divide by n
3	−1.4710	−0.2065	−1.3333
5	−0.7311	+0.0861	−0.8000
10	−0.4173	−0.0193	−0.4000
30	−0.1776	−0.0457	−0.1333
100	−0.0496	−0.0097	−0.0400

(d) Is the sample standard deviation unbiased?

No. Although the sample variance is unbiased, taking its square root does not preserve unbiasedness.

n	Mean sample SD	Estimated bias: mean minus 2
3	1.7131	−0.2869
5	1.9018	−0.0982
10	1.9392	−0.0608
30	1.9723	−0.0277
100	1.9928	−0.0072

The sample SD underestimates 2 on average, with the bias approaching zero as sample size grows. The square-root transformation is curved: the average of square roots is generally smaller than the square root of the average. Thus, an unbiased estimate of variance does not automatically yield an unbiased estimate of standard deviation.

(e)

```
# Note: These SDs describe variation ACROSS the 1,000 variance estimates.
variability = data.frame(
  estimator = c("Divide by n", "Divide by n - 1"),
  SD_of_estimates = c(sd(results5$variance_n), sd(results5$variance_n1)),
  bias = c(mean(results5$variance_n) - 4, mean(results5$variance_n1) - 4),
  MSE = c(mean((results5$variance_n - 4)^2),
# MSE averages squared distances from the TRUE variance. It includes bias and variation.
          mean((results5$variance_n1 - 4)^2)))
cat("\nPROBLEM 1(e): variability, bias, and mean squared error at n = 5\n")


PROBLEM 1(e): variability, bias, and mean squared error at n = 5

print(variability, row.names = FALSE)

       estimator SD_of_estimates        bias      MSE
     Divide by n        2.306722 -0.73114574 5.850218
 Divide by n - 1        2.883402  0.08606782 8.313101
```

Comparing variability at n = 5

Estimator	SD across the 1,000 variance estimates	Estimated bias	Mean squared error
Divide by n	2.3067	−0.7311	5.8502
Divide by n − 1	2.8834	+0.0861	8.3131

The divide-by-n estimator is less variable. At n = 5, every divide-by-n estimate is exactly 4/5 of its corresponding corrected estimate. Its sampling SD is therefore also exactly 4/5 as large.

Less variability alone does not establish that an estimator is better: an estimator can repeatedly give similar answers that are systematically too low.

If "better" means unbiased, divide by n − 1 is preferable. If "better" means lower mean squared error (MSE), the divide-by-n estimator is actually better in this normal-population example at n = 5. Thus, neither estimator is best under every criterion.

Problem 2: Omitted variable bias

Given: The true equation is y = 1 + 2x + 3z + error, where x is news consumption, z is political interest, and y is political knowledge.

(a) Generating 100 observations with rho = 0.5

```
n = 100
rho = 0.5
set.seed(60204)

# Separate random draws make z, u, and error independent in the Data Generation Process.
z = rnorm(n, mean = 0, sd = 1)
u = rnorm(n, mean = 0, sd = 1)
error = rnorm(n, mean = 0, sd = 1)
x = rho * z + sqrt(1 - rho^2) * u
y = 1 + 2 * x + 3 * z + error
d = data.frame(y, x, z, u, error)

# Means and SDs are found to be close to population values, not exactly equal.
checks = data.frame(variable = c("z", "u", "error", "x", "y"),
                    mean = c(mean(z), mean(u), mean(error), mean(x), mean(y)),
                    sd = c(sd(z), sd(u), sd(error), sd(x), sd(y)))
print(checks, row.names = FALSE)
```

```
 variable        mean        sd
        z -0.04729414 0.8900707
        u  0.12287904 0.9756262
    error -0.11592533 0.9659683
        x  0.08276930 0.9537088
        y  0.90773085 3.9462094
```

```
cat("Correlation matrix (finite-sample correlations are approximate):\n")
```

```
Correlation matrix (finite-sample correlations are approximate):
```

```
print(round(cor(d), 3))
```

```
          y      x      z      u  error
y     1.000  0.764  0.884  0.396  0.133
x     0.764  1.000  0.464  0.884 -0.137
z     0.884  0.464  1.000 -0.003 -0.068
u     0.396  0.884 -0.003  1.000 -0.119
error 0.133 -0.137 -0.068 -0.119  1.000
```

The code generates independent standard-normal draws for z, u, and error, then constructs x and y exactly as specified.

Variable	Sample mean	Sample SD
z	−0.0473	0.8901
u	0.1229	0.9756
error	−0.1159	0.9660
x	0.0828	0.9537
y	0.9077	3.9462

The sample correlation of x and z is 0.4638, reasonably close to the population correlation of 0.5.

(b) Fit both models and plot y against x

```
# Use the dataset generated in part (a).
omitted_model = lm(y ~ x, data = d)
full_model = lm(y ~ x + z, data = d)
cat("Model omitting political interest z:\n")
```

```
Model omitting political interest z:
```

```
print(summary(omitted_model))
```

```

Call:
lm(formula = y ~ x, data = d)

Residuals:
    Min      1Q  Median      3Q     Max
-6.3778 -1.9150  0.1034  1.6659  7.8966

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   0.6462     0.2570   2.514   0.0136 *
x             3.1600     0.2698  11.711   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.561 on 98 degrees of freedom
Multiple R-squared:  0.5832,    Adjusted R-squared:  0.579
F-statistic: 137.1 on 1 and 98 DF,  p-value: < 2.2e-16
```

```
cat("Model including political interest z:\n")
```

```
Model including political interest z:
```

```
print(summary(full_model))
```

```

Call:
lm(formula = y ~ x + z, data = d)

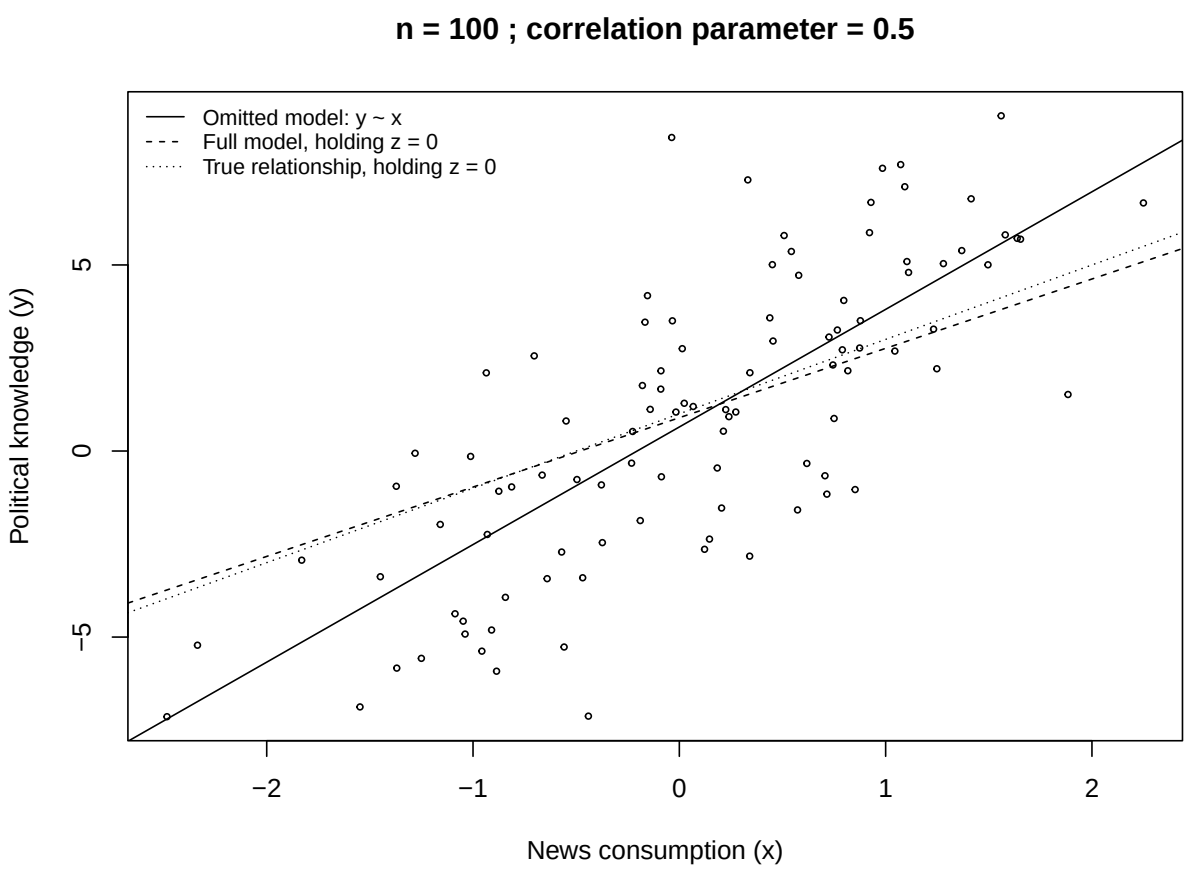
Residuals:
     Min       1Q   Median       3Q      Max
-2.47495 -0.68840 -0.02392  0.67994  2.40088

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.89507    0.09757   9.173 8.18e-15 ***
x            1.86386    0.11499  16.209  < 2e-16 ***
z            2.99420    0.12321  24.302  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 0.9667 on 97 degrees of freedom
Multiple R-squared:  0.9412,    Adjusted R-squared:   0.94
F-statistic: 776.4 on 2 and 97 DF,  p-value: < 2.2e-16

plot(x, y, pch = 1, cex = 0.5,
     xlab = "News consumption (x)", ylab = "Political knowledge (y)",
     main = paste("n =", n, "; correlation parameter =", rho))
abline(omitted_model)

# For the full model and true relationship, hold z fixed at 0.
abline(a = coef(full_model)["(Intercept)"],
       b = coef(full_model)["x"], lty = 2)
abline(a = 1, b = 2, lty = 3)
legend("topleft", legend = c("Omitted model: y ~ x",
       "Full model, holding z = 0", "True relationship, holding z = 0"),
       lty = c(1, 2, 3), bty = "n", cex = 0.8)
```



Model	Estimated coefficient on x	Standard error
y ~ x	3.1600	0.2698
y ~ x + z	1.8639	0.1150

The model omitting political interest overstates the effect of news consumption in this sample. The full model's estimate is much closer to 2. Its estimated coefficient on z is 2.9942, close to the true value of 3.

Political interest increases political knowledge and is positively associated with news consumption. When z is omitted, the regression attributes some of political interest's contribution to x.

The scatterplot shows a positive relationship. The solid line is the simple regression. The dashed line shows the multiple regression holding z = 0; the dotted line shows the true equation holding z = 0.

(c) Increasing the sample size to 1,000, keeping rho = 0.5

```
# Repeating parts (a) and (b) with 1,000 observations.
n = 1000
rho = 0.5
set.seed(60205)

# Separating random draws make z, u, and error independent in the Data Generation Process.
z = rnorm(n, mean = 0, sd = 1)
u = rnorm(n, mean = 0, sd = 1)
error = rnorm(n, mean = 0, sd = 1)
x = rho * z + sqrt(1 - rho^2) * u
y = 1 + 2 * x + 3 * z + error
d = data.frame(y, x, z, u, error)

# Means and SDs are found to be close to population values, not exactly equal.
checks = data.frame(variable = c("z", "u", "error", "x", "y"),
                    mean = c(mean(z), mean(u), mean(error), mean(x), mean(y)),
                    sd = c(sd(z), sd(u), sd(error), sd(x), sd(y)))
print(checks, row.names = FALSE)

 variable        mean        sd
        z  0.03123640 0.9838415
        u  0.06101701 0.9877564
    error -0.02664725 0.9734476
        x  0.06846048 1.0010795
        y  1.20398290 4.4499808

cat("Correlation matrix (finite-sample correlations are approximate):\n")

Correlation matrix (finite-sample correlations are approximate):

print(round(cor(d), 3))

          y     x      z     u  error
y     1.000 0.799  0.894 0.421  0.217
x     0.799 1.000  0.520 0.871  0.017
z     0.894 0.520  1.000 0.034 -0.014
u     0.421 0.871  0.034 1.000  0.029
error 0.217 0.017 -0.014 0.029  1.000

omitted_model = lm(y ~ x, data = d)
full_model = lm(y ~ x + z, data = d)
cat("Model omitting political interest z:\n")

Model omitting political interest z:
```

```
print(summary(omitted_model))
```

```

Call:
lm(formula = y ~ x, data = d)

Residuals:
    Min      1Q  Median      3Q     Max
-7.6637 -1.8363 -0.1014  1.9645  7.9600

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.96089    0.08490   11.32   <2e-16 ***
x            3.55086    0.08465   41.95   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.678 on 998 degrees of freedom
Multiple R-squared:  0.6381,    Adjusted R-squared:  0.6377
F-statistic:  1760 on 1 and 998 DF,  p-value: < 2.2e-16
```

```
cat("Model including political interest z:\n")
```

```
Model including political interest z:
```

```
print(summary(full_model))
```

```

Call:
lm(formula = y ~ x + z, data = d)

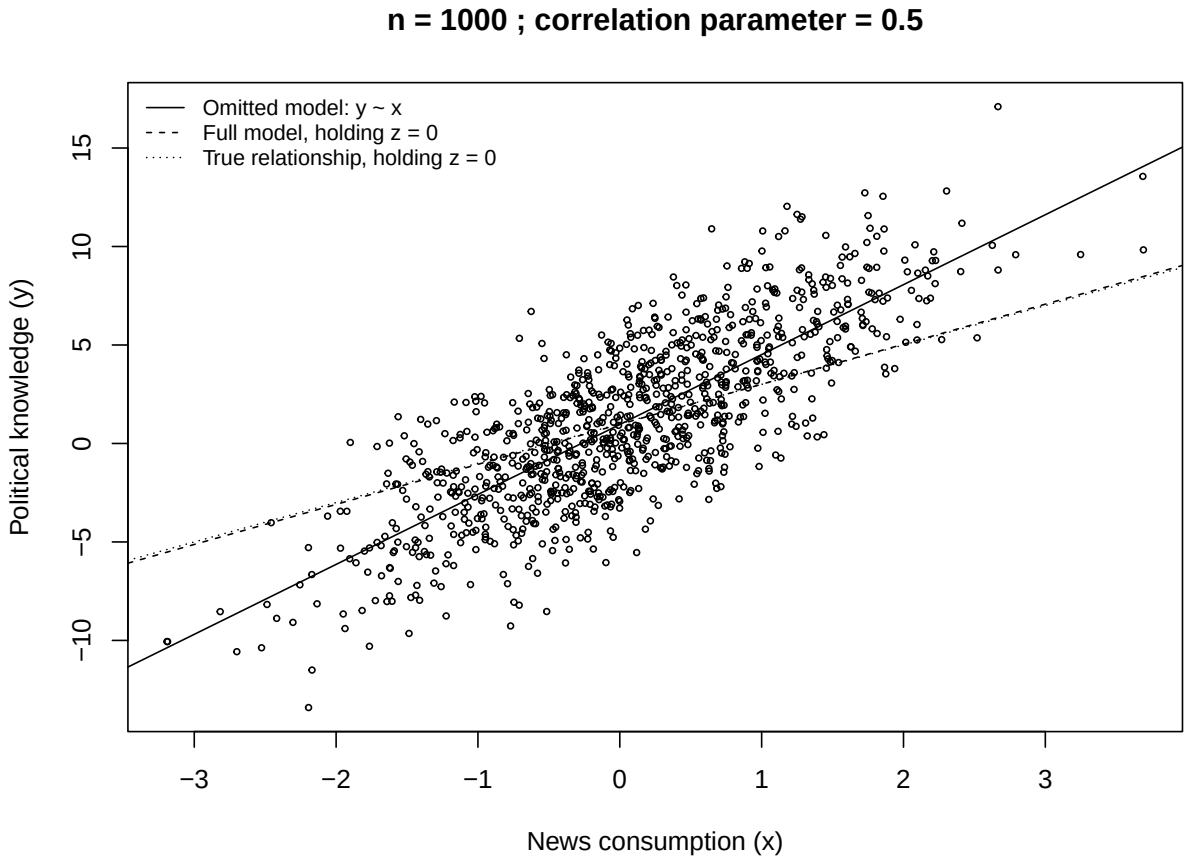
Residuals:
     Min       1Q   Median       3Q      Max
-2.86462 -0.64862 -0.00137  0.68515  2.73655

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.97207    0.03087   31.49   <2e-16 ***
x            2.03320    0.03604   56.41   <2e-16 ***
z            2.96826    0.03667   80.94   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9739 on 997 degrees of freedom
Multiple R-squared:  0.9522,    Adjusted R-squared:  0.9521
F-statistic:  9930 on 2 and 997 DF,  p-value: < 2.2e-16
```

```
plot(x, y, pch = 1, cex = 0.5,
     xlab = "News consumption (x)", ylab = "Political knowledge (y)",
     main = paste("n =", n, "; correlation parameter =", rho))
abline(omitted_model)
```

```
# For the full model and true relationship, holding z fixed at 0.
abline(a = coef(full_model)["(Intercept)"],
       b = coef(full_model)["x"], lty = 2)
abline(a = 1, b = 2, lty = 3)
legend("topleft", legend = c("Omitted model: y ~ x",
       "Full model, holding z = 0", "True relationship, holding z = 0"),
       lty = c(1, 2, 3), bty = "n", cex = 0.8)
```



Model	Estimated coefficient on x	Standard error
y ~ x	3.5509	0.0846
y ~ x + z	2.0332	0.0360

More data did not resolve the omitted-variable bias. The omitted-model estimate is now close to 3.5, whereas the full-model estimate is close to 2. Both standard errors are smaller than in the 100-observation dataset.

Larger samples reduce random sampling uncertainty, but do not correct the omitted model's systematic error. That model becomes more precise around the wrong target.

(d) Use 1,000 observations with rho = 0

```
# Repeating part (c) with rho = 0.
```

```
n = 1000
rho = 0
set.seed(60206)

# Separating random draws, making z, u, and error independent in the Data Generation Process.
z = rnorm(n, mean = 0, sd = 1)
u = rnorm(n, mean = 0, sd = 1)
error = rnorm(n, mean = 0, sd = 1)
x = rho * z + sqrt(1 - rho^2) * u
y = 1 + 2 * x + 3 * z + error
d = data.frame(y, x, z, u, error)

# Means and SDs are found to be close to population values, not exactly equal.
checks = data.frame(variable = c("z", "u", "error", "x", "y"),
                    mean = c(mean(z), mean(u), mean(error), mean(x), mean(y)),
                    sd = c(sd(z), sd(u), sd(error), sd(x), sd(y)))
print(checks, row.names = FALSE)

 variable       mean        sd
        z 0.01233742 0.9579408
        u 0.01647475 0.9996191
    error 0.01315506 1.0175711
        x 0.01647475 0.9996191
        y 1.08311680 3.6830746

cat("Correlation matrix (finite-sample correlations are approximate):\n")

Correlation matrix (finite-sample correlations are approximate):

print(round(cor(d), 3))

          y      x     z      u  error
y     1.000  0.550 0.795  0.550  0.295
x     0.550  1.000 0.012  1.000 -0.008
z     0.795  0.012 1.000  0.012  0.029
u     0.550  1.000 0.012  1.000 -0.008
error 0.295 -0.008 0.029 -0.008  1.000

omitted_model = lm(y ~ x, data = d)
full_model = lm(y ~ x + z, data = d)
cat("Model omitting political interest z:\n")

Model omitting political interest z:

print(summary(omitted_model))


Call:
lm(formula = y ~ x, data = d)

Residuals:
    Min      1Q  Median      3Q     Max
```

```
-9.7940 -2.0898  0.0613  2.0269  9.3279

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.04974    0.09735   10.78   <2e-16 ***
x            2.02587    0.09742   20.80   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.078 on 998 degrees of freedom
Multiple R-squared:  0.3023,    Adjusted R-squared:  0.3016
F-statistic: 432.5 on 1 and 998 DF,  p-value: < 2.2e-16


cat("Model including political interest z:\n")

Model including political interest z:

print(summary(full_model))



Call:
lm(formula = y ~ x + z, data = d)

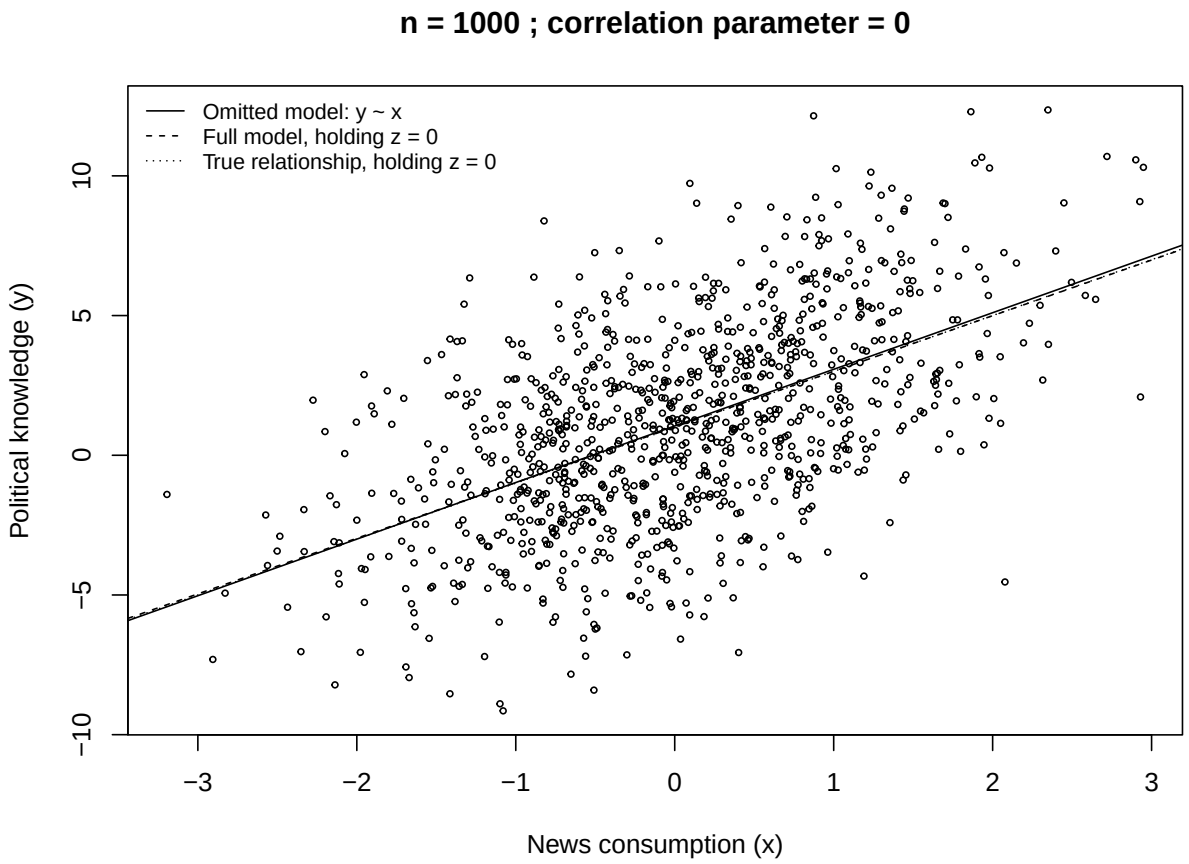
Residuals:
    Min      1Q  Median      3Q     Max
-3.2910 -0.6498 -0.0173  0.6727  3.0728

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  1.01292    0.03220   31.45   <2e-16 ***
x            1.99129    0.03223   61.79   <2e-16 ***
z            3.03105    0.03363   90.13   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.018 on 997 degrees of freedom
Multiple R-squared:  0.9237,    Adjusted R-squared:  0.9236
F-statistic:  6038 on 2 and 997 DF,  p-value: < 2.2e-16


plot(x, y, pch = 1, cex = 0.5,
     xlab = "News consumption (x)", ylab = "Political knowledge (y)",
     main = paste("n =", n, "; correlation parameter =", rho))
abline(omitted_model)

# For the full model and true relationship, holding z fixed at 0.
abline(a = coef(full_model)["(Intercept)"],
       b = coef(full_model)["x"], lty = 2)
abline(a = 1, b = 2, lty = 3)
legend("topleft", legend = c("Omitted model: y ~ x",
       "Full model, holding z = 0", "True relationship, holding z = 0"),
       lty = c(1, 2, 3), bty = "n", cex = 0.8)
```



Model	Estimated coefficient on x	Standard error
y ~ x	2.0259	0.0974
y ~ x + z	1.9913	0.0322

Both estimates are now close to 2. With rho = 0, x equals u and is independent of z in this process. Political interest still affects knowledge, but it is no longer associated with news consumption.

Including z still improves precision here. It explains variation in y that would otherwise be left in the error term, reducing the standard error for x from 0.0974 to 0.0322. On the plot, the fitted lines are close to the true line.

The modeling lesson is to use substantive theory about the relationships among variables. In this additive setup, omitting z biases x's coefficient when z both affects y and is correlated with x. An omitted predictor of y does not automatically bias the coefficient on x. We should account for relevant confounders rather than expect a larger dataset to fix their omission.